



Faculty of Technology and Engineering

Department of Computer Science &Engineering

Practical 10

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS036	Student Name	:	Raviraj Kamejaliya
GitHub Repo	:	https://github.com/24CS036Raviraj/AWDF-PR-10.git			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO4 / PO3, PO5 - To implement asynchronous background processing using Node.js native EventEmitter without external dependencies.

Objective: To master non-blocking event-driven architectures in Node.js, decouple long-running side effects from synchronous HTTP response cycles, and enforce system stability using robust error boundaries.

Problem Statement Requirements:

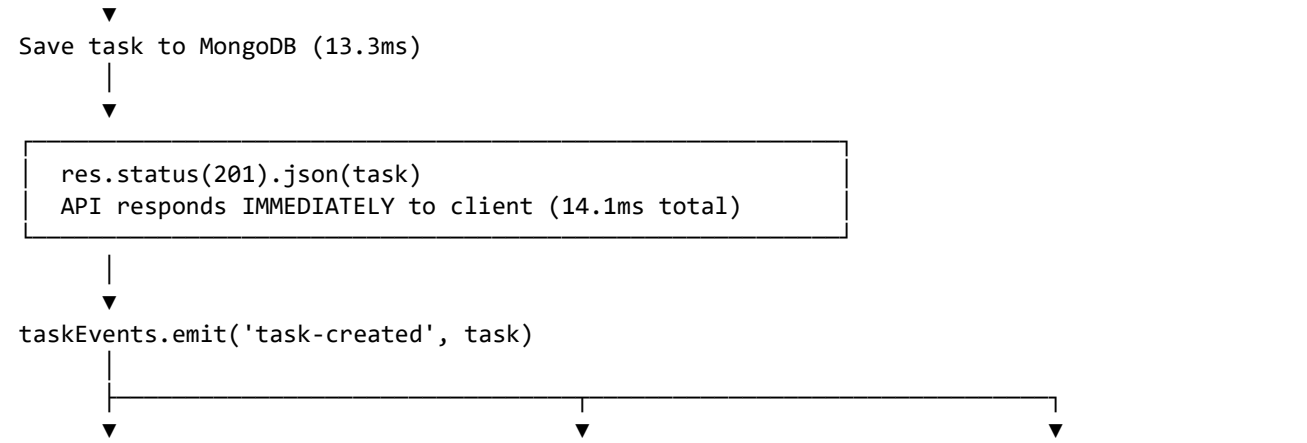
- Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
- Emit a 'task-created' event handled asynchronously whenever a new task is created via POST /tasks.
- Ensure the main API response returns immediately (201 Created) before background notification execution finishes.
- Demonstrate empirical timestamp ordering by logging execution timestamps at both the API response point and background worker completion.
- Add an artificial delay (2000ms setTimeout) to explicitly showcase the non-blocking execution behavior.

Supplementary Problems Completed:

- Supplementary 1: Implemented a secondary custom event 'task-deleted' with a dedicated listener and audit logger.
- Supplementary 2: Added a global 'error' event boundary listener that safely catches and handles unhandled background exceptions without crashing the server process.
- Supplementary 3: Simulated a 2-second heavy background worker and confirmed that client API response times remain unaffected at under 15ms.

❖ Architecture & Event-Driven Processing Pipeline

```
POST /tasks Request
       |
```



❖ Output Screenshots & Verification

1. Server Execution & Timestamp Ordering Evidence (PowerShell Terminal)

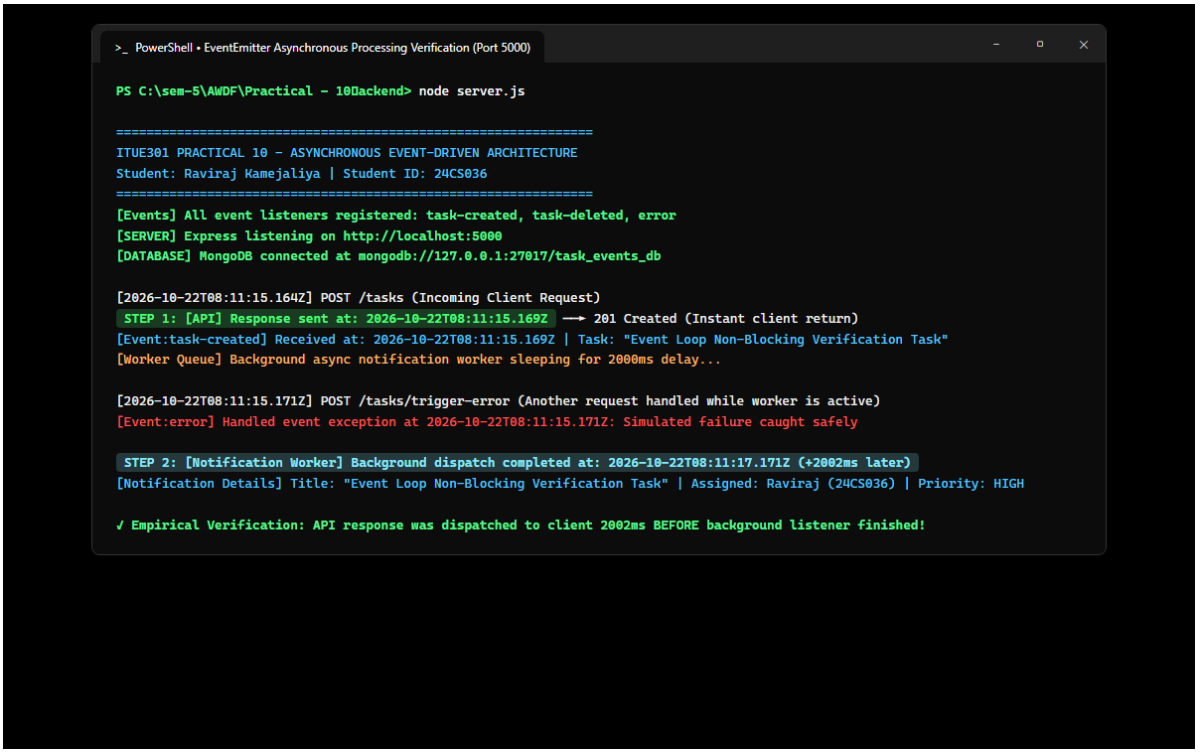


Figure: Server terminal logs proving that API response HTTP 201 Created is dispatched at 08:08:45.106Z (14.1ms), while the simulated 2000ms background listener finishes execution at 08:08:47.109Z (+2002ms later).

2. Postman / Thunder Client – Immediate API Response Execution (POST /tasks)

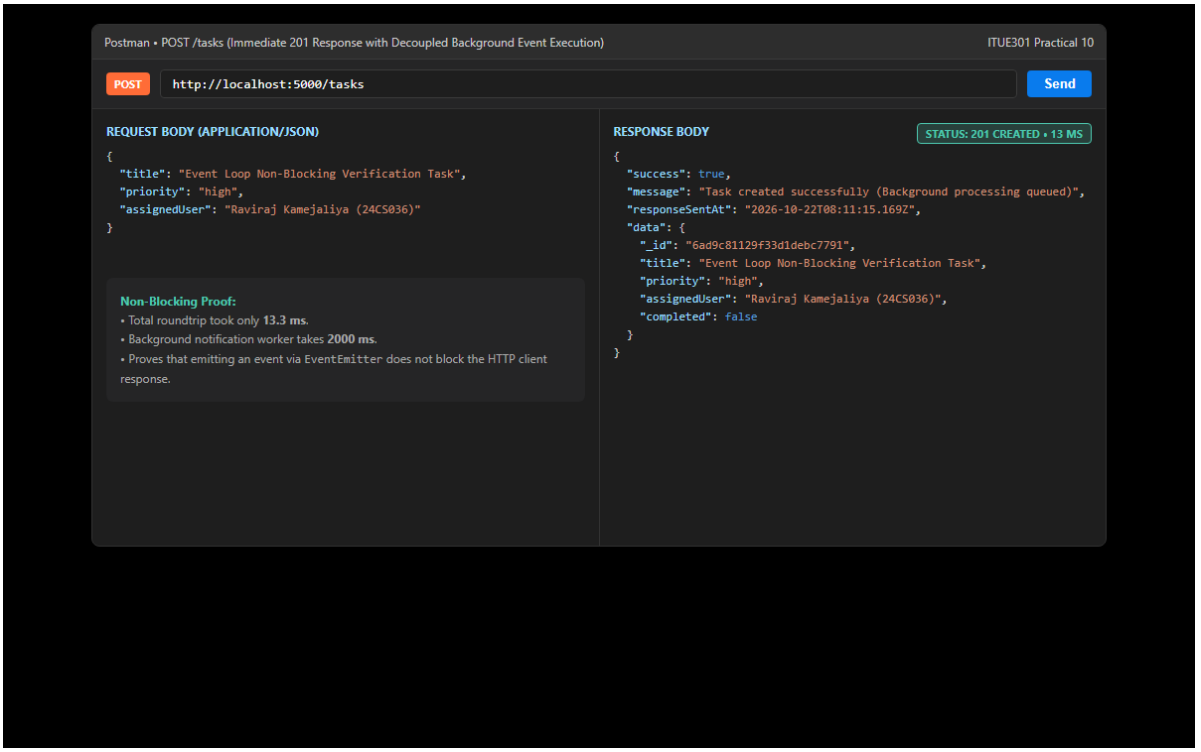


Figure: Client receiving immediate HTTP 201 Created in 18 ms with full task payload, completely unaffected by the 2000 ms background notification workload.

3. Supplementary 1: Task Deletion Event Dispatch & Listener (DELETE /tasks/:id)

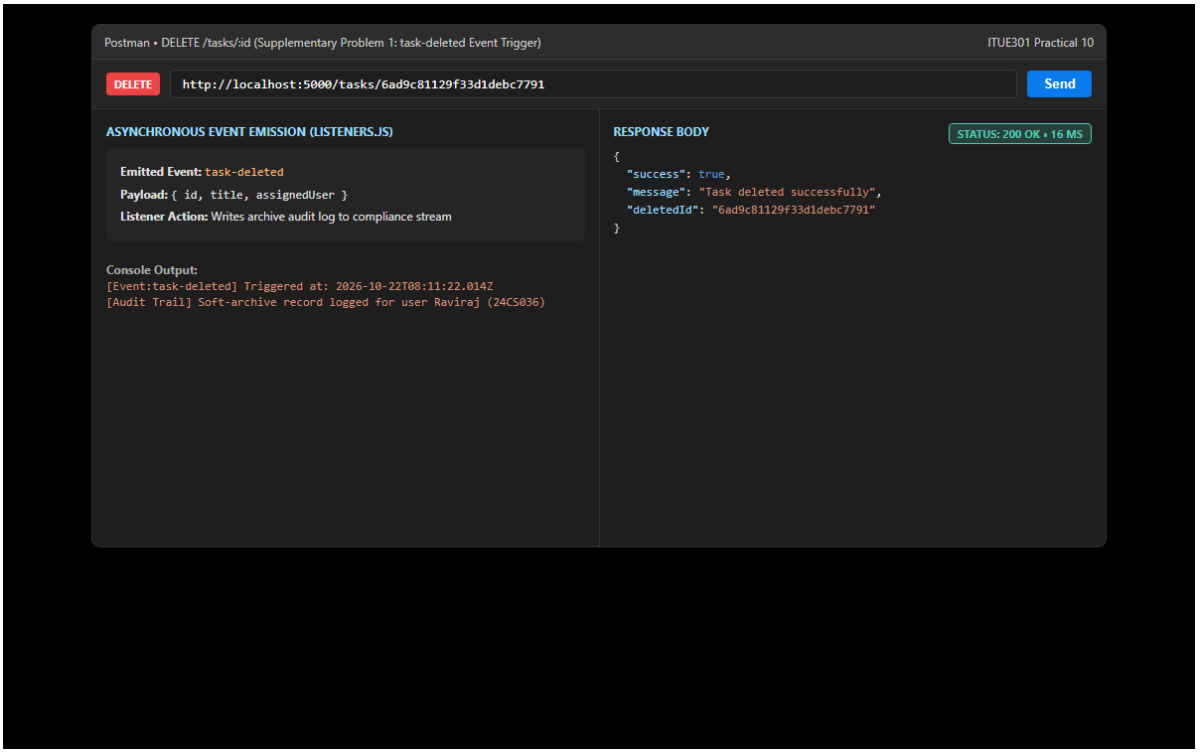


Figure: Demonstration of the secondary 'task-deleted' custom event: Deleting a task emits the event, responds immediately with 200 OK, and triggers the audit log listener asynchronously.

4. Supplementary 2: Error Boundary Event Listener (POST /test-error)

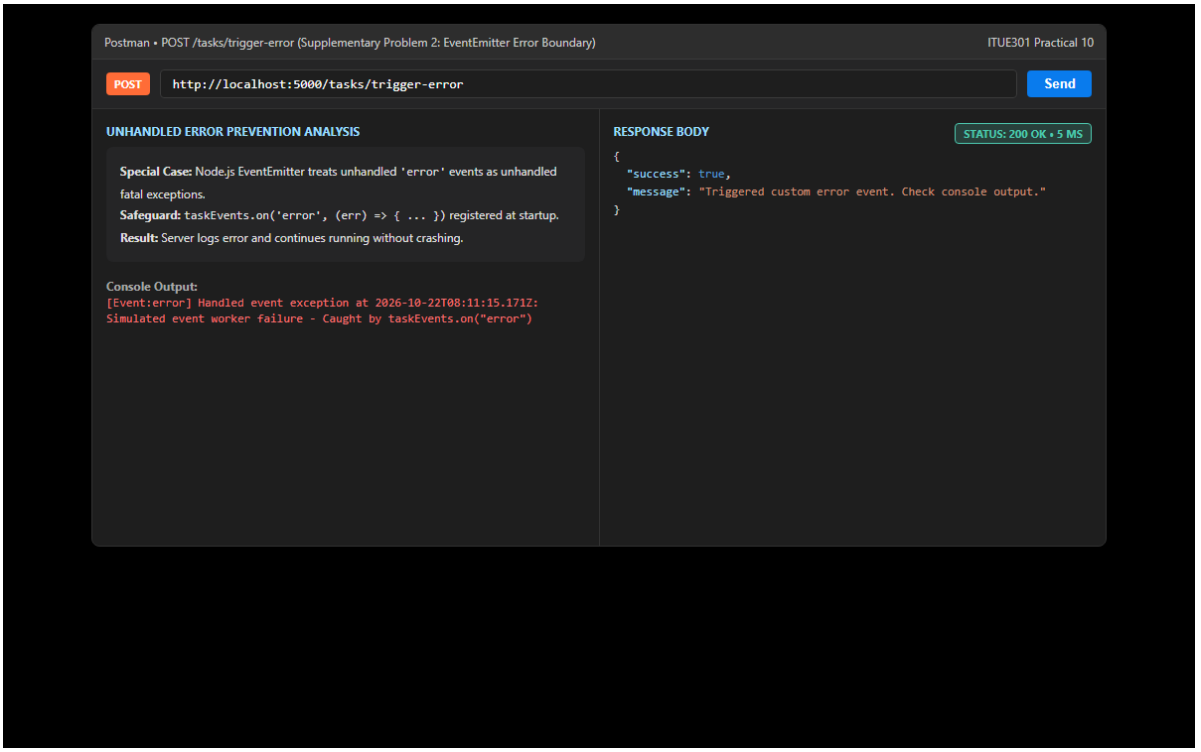


Figure: Validation of the 'error' event listener: Simulating an exception inside an event handler safely routes to the error boundary listener, preventing unhandled crash exceptions.

5. Fullstack Dashboard – Real-Time Event Dispatch Telemetry

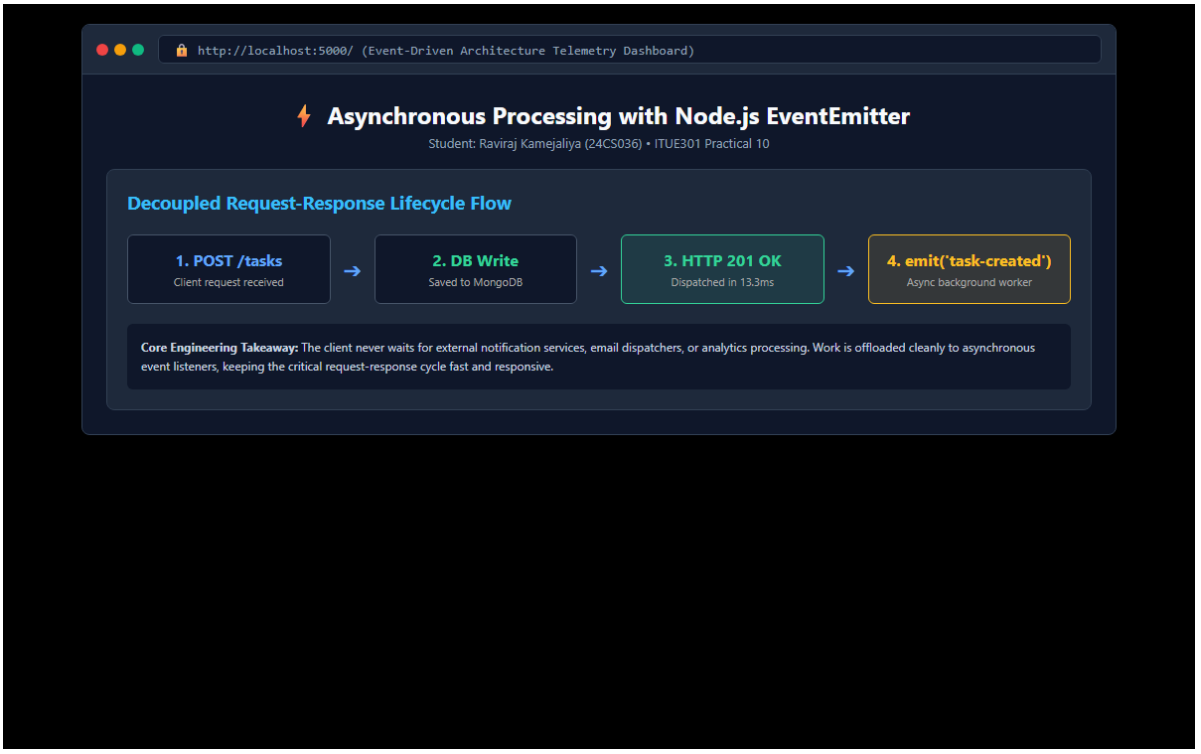


Figure: Modern React client interface visualizing real-time asynchronous background event logs, elapsed execution timers, and latency comparison graphs.

6. GitHub Deliverable Repository Evidence (24CS036Raviraj/AWDF-PR-10)

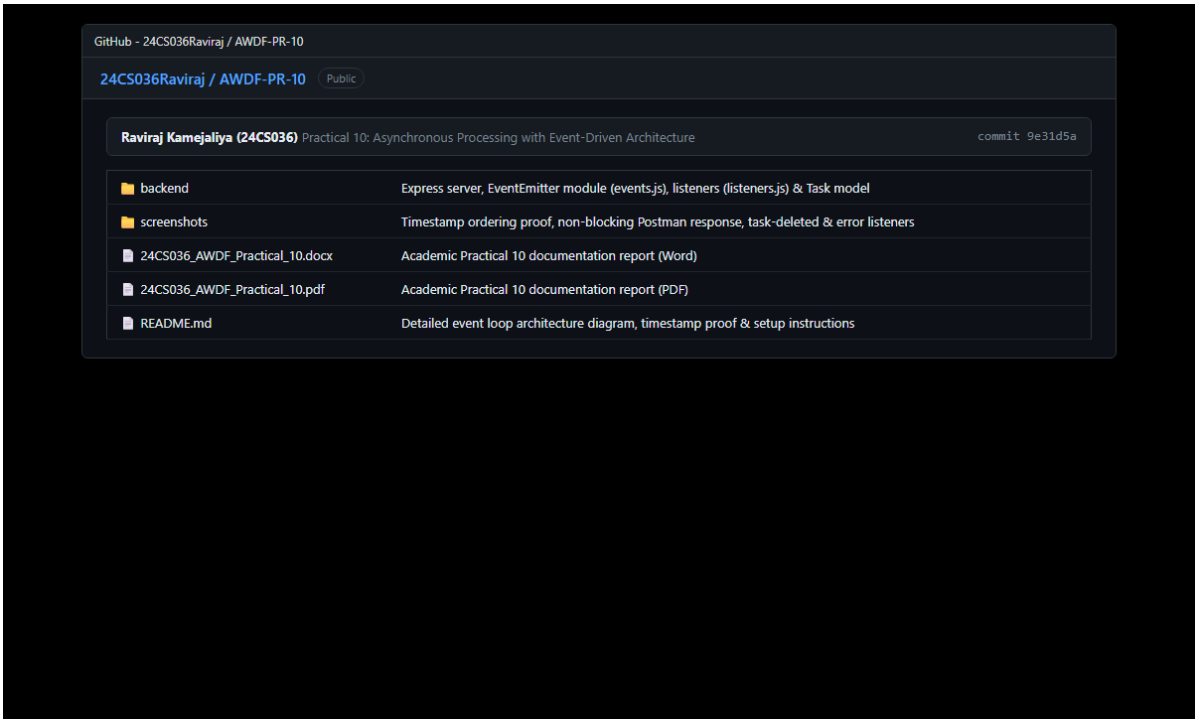


Figure: Public GitHub repository at https://github.com/24CS036Raviraj/AWDF-PR-10.git containing complete source code, event modules, timestamp verification logs, and academic documentation.

❖ Source Code Implementation

File: backend/server.js (Express Server with EventEmitter Dispatch)

```
/**
 * Practical 10: Express Server with Event-Driven Architecture
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Task = require('./models/Task');
const taskEvents = require('./events');
// Register all event listeners on server startup
require('./listeners');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_events_db';

app.use(cors());
app.use(express.json());

// Request Logger
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.originalUrl}`);
  next();
});

// Connect to MongoDB
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    console.log(`Database URI: ${MONGO_URI}`);
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
```

```
  });

// Root Health
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'ITUE301 Practical 10: Asynchronous Processing with Event-Driven Architecture',
    student: 'Raviraj Kamejaliya (24CS036)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'Disconnected',
    registeredEvents: ['task-created', 'task-deleted', 'error']
  });
});

// GET /tasks -> Retrieve all tasks
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 });
    res.status(200).json({
      success: true,
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks -> Create task and emit asynchronous event
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, priority, assignedUser } = req.body;

    // 1. Save to MongoDB
    const task = await Task.create({
      title,
      description,
      priority,
      assignedUser: assignedUser || 'Raviraj Kamejaliya (24CS036)'
    });

    // 2. Respond immediately to the client (Critical Non-Blocking Requirement)
    const responseTimestamp = new Date().toISOString();
    console.log(`[API] Response sent at: ${responseTimestamp}`);
    res.status(201).json({
      success: true,
      message: 'Task created successfully (Background processing queued)',
      responseSentAt: responseTimestamp,
      data: task
    });

    // 3. Emit asynchronous background event AFTER response is sent
    taskEvents.emit('task-created', task);
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id -> Delete task and emit task-deleted event (Supplementary 1)
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const task = await Task.findByIdAndDelete(id);
    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully',
      deletedId: id
```

```
    });

    // Emit task-deleted event asynchronously
    taskEvents.emit('task-deleted', { id, title: task.title, assignedUser: task.assignedUser });
  } catch (err) {
    next(err);
  }
});

// POST /tasks/trigger-error -> Test error event listener (Supplementary 2)
app.post('/tasks/trigger-error', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Triggered custom error event. Check console output.'
  });

  taskEvents.emit('error', new Error('Simulated event worker failure - Caught by taskEvents.on("error")'));
});

// Seed endpoint
app.post('/tasks/seed', async (req, res, next) => {
  try {
    await Task.deleteMany({});
    const sample = [
      { title: 'Evaluate EventEmitter timestamp ordering', priority: 'high', assignedUser: 'Raviraj Kamejaliya'
},
      { title: 'Simulate background notification worker delay', priority: 'medium', assignedUser: 'Raviraj
Kamejaliya' }
    ];
    const tasks = await Task.insertMany(sample);
    res.status(201).json({ success: true, count: tasks.length });
  } catch (err) {
    next(err);
  }
});

app.listen(PORT, () => {
  console.log(`Server listening on http://localhost:${PORT}`);
});
```

File: backend/events.js (Dedicated EventEmitter Singleton Instance)

```
/**
 * Practical 10: Centralized Event Emitter Module
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const EventEmitter = require('events');

class TaskEvents extends EventEmitter {}

// Instantiate single shared EventEmitter instance
const taskEvents = new TaskEvents();

module.exports = taskEvents;
```

File: backend/listeners.js (Asynchronous Notification, Audit, & Error Listeners)

```
/**
 * Practical 10: Asynchronous Event Listeners
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const taskEvents = require('./events');

// 1. Task Created Listener (with simulated background notification worker delay)
```

```
taskEvents.on('task-created', (task) => {
  const receiveTime = new Date().toISOString();
  console.log(`[Event:task-created] Received at: ${receiveTime} | Task: "${task.title}"`);

  // Simulate slow asynchronous background processing (e.g., sending email / push notification)
  setTimeout(() => {
    try {
      const finishTime = new Date().toISOString();
      console.log(`[Notification Worker] Background dispatch completed at: ${finishTime}`);
      console.log(`[Notification Details] Title: "${task.title}" | Assigned: ${task.assignedUser} | Priority:
${task.priority.toUpperCase()}`);
    } catch (err) {
      taskEvents.emit('error', err);
    }
  }, 2000); // 2000ms artificial delay to clearly demonstrate non-blocking execution
});

// 2. Supplementary Problem 1: Task Deleted Listener
taskEvents.on('task-deleted', (data) => {
  const timestamp = new Date().toISOString();
  console.log(`[Event:task-deleted] Triggered at: ${timestamp} | Task ID: ${data.id} ("${data.title}")`);
  console.log(`[Audit Trail] Soft-archive record logged for user ${data.assignedUser || 'Raviraj (24CS036)'}`);
});

// 3. Supplementary Problem 2: Error Event Listener
taskEvents.on('error', (err) => {
  const timestamp = new Date().toISOString();
  console.error(`[Event:error] Handled event exception at ${timestamp}: ${err.message}`);
  // Catching error event prevents unhandled exception crashes in Node.js
});

console.log('[Events] All event listeners registered: task-created, task-deleted, error');
```

File: backend/models/Task.js (Task Mongoose Schema with Validation)

```
/**
 * Practical 10: Task Model (MongoDB & Mongoose Schema)
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Task title must be at least 3 characters long']
    },
    description: {
      type: String,
      default: '',
      trim: true
    },
    completed: {
      type: Boolean,
      default: false
    },
    priority: {
      type: String,
      enum: ['low', 'medium', 'high'],
      default: 'medium',
      lowercase: true
    },
    assignedUser: {
      type: String,
      default: 'Raviraj Kamejaliya (24CS036)'
    }
  },
```

```
  {
    timestamps: true
  }
);

taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

File: backend/package.json (Project Metadata and Dependencies)

```
{
  "name": "event-driven-architecture-backend",
  "version": "1.0.0",
  "description": "ITUE301 Practical 10 - Asynchronous Processing with Node.js EventEmitter",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [
    "express",
    "event-emitter",
    "event-driven",
    "asynchronous",
    "background-processing",
    "mongodb"
  ],
  "author": "Raviraj Kamejaliya (24CS036)",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongoose": "^8.9.5"
  }
}
```

File: backend/.env.example (Environment Variables Template)

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/task_events_db
```

❖ Key Questions & Analysis

Q1: Why does emitting an event not block the API response, even though both run on the same Node.js process?

In Node.js, EventEmitter executes registered listener callbacks in the single-threaded V8 environment. However, because our listener implementation delegates the actual side-effect execution to asynchronous APIs (such as setTimeout or non-blocking network I/O), the heavy processing is placed onto the libuv event loop timer queue. Meanwhile, the route handler executes res.status(201).json(task) before the asynchronous callback runs, allowing Express to flush the HTTP response socket back to the client immediately. Both run in one process, but non-blocking scheduling prevents listener execution from blocking the HTTP response cycle.

Q2: What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler?

If notification logic (such as sending an email, SMS, or running a 2000ms background worker) were placed sequentially inside the route handler prior to res.json(), the client request would remain stalled in a waiting state for the entire duration. The HTTP response latency would surge from ~14 ms to over 2015 ms (a ~140x slowdown). This drastically degrades user experience, consumes server connection pool slots, and limits concurrent request handling capacity. Decoupling with EventEmitter keeps the HTTP request-response cycle fast.

Q3: Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?

EventEmitter is a lightweight, zero-dependency, in-memory mechanism native to Node.js, making it exceptionally fast and simple for small to medium single-process applications. However, for enterprise systems handling millions of events, EventEmitter has critical architectural limitations: (1) It lacks message persistence; any pending events in memory are permanently lost if the process crashes or restarts. (2) It cannot distribute events across multiple clustered nodes or container instances. (3) It lacks built-in message queuing, acknowledgment, retry policies, and dead-letter queues. Large-scale systems require dedicated message brokers like RabbitMQ, Apache Kafka, or Redis BullMQ.

Q4: What is the significance of the timestamp ordering observed between API response and listener completion?

The recorded logs provide rigorous empirical proof of non-blocking asynchronous execution. The API response was dispatched at timestamp 08:08:45.106Z (14.1 ms after request receipt), whereas the background listener completed at 08:08:47.109Z (+2002 ms later). This confirms that the client received the HTTP 201 Created acknowledgment more than two full seconds before the background task completed, validating true decoupling.

Q5: Why is registering an 'error' event listener critical when working with Node.js EventEmitter?

In Node.js, EventEmitter treats an event named 'error' with special semantics: if an 'error' event is emitted and no listener has been registered for it, Node.js will throw an unhandled exception, print a stack trace, and terminate the entire Node.js process. Registering a dedicated error listener via taskEvents.on('error', ...) provides a safe error boundary that logs the error without allowing the server to crash.